Monday, February 14, 2022 2:30 AM

1. **Depth-first search**

- Graph G = (V , E),either directed or undirected, (no source vertex)
- Output: d [v] : discovery time, f [v] : finishing time for v ∈ V
- methodically explore every edge
- Start over from different vertices as necessary
- As soon as we discover a vertex, explore from it
- As DFS progresses, every vertex has a color:
- WHITE: undiscovered
- GRAY: discovered, not finished
- BLACK: finished

2. **Depth-first search**

Algorithm 1
DFS(V, E)

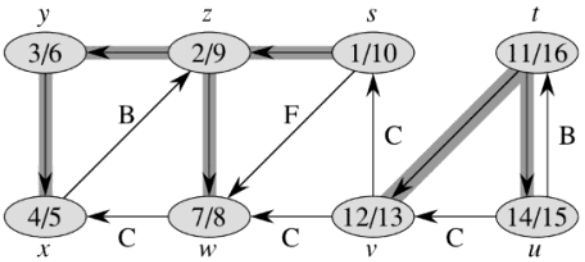
Input: A graph
G:=(V,E)

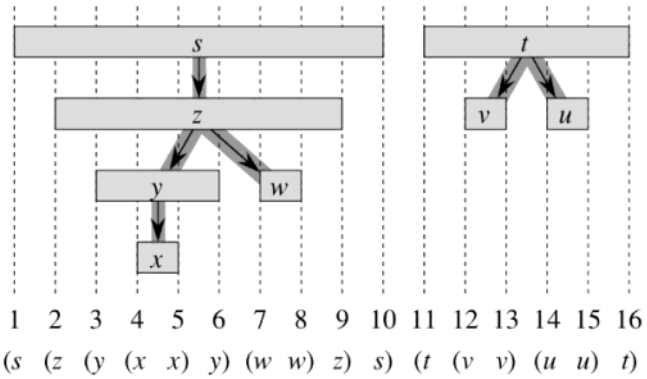
Output:
d[v], f[v] for all v∈V

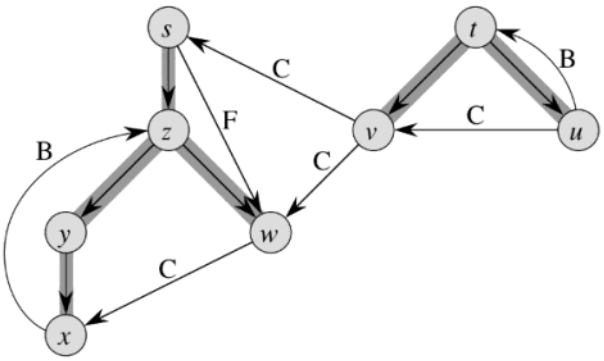
```
Color all vertex WHITE
time ← 0
for each v ∈ V do
    if color[u] = WHITE then DFS-VISIT(u)
end for

DFS-VISIT(u)
color[u]←GRAY, time=time+1,d[u]←time
for each v ∈ Adj[u] do
    if color[v] = WHITE then DFS-VISIT(v)
end for
color[u]←BLACK ,time=time+1,f[u]←time
```

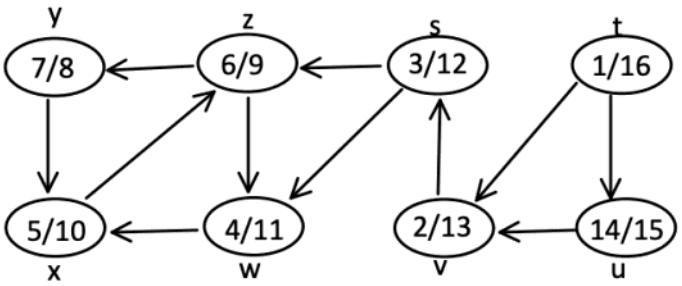
- Example 1: Start from s

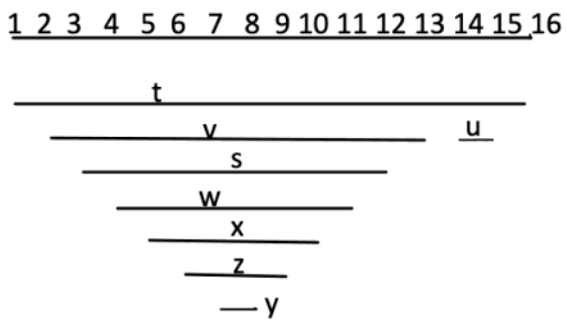


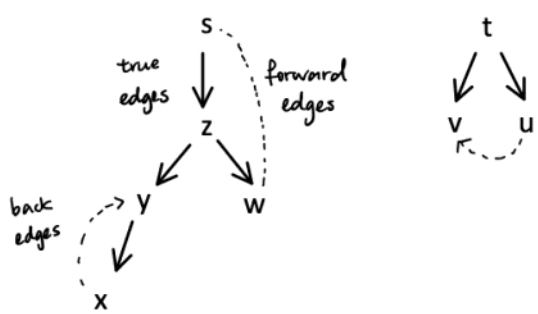




- Example 2: Start from t







3. Correctness

- **Parenthesis theorem**
  Exactly one of the following holds:
- u and v are not descendant and

  d[u]<f[u]<d[v]<f[v]
  or

  d[v]<f[v]<d[u]<f[u]

- *v descendant u back-edge*

  d[u]<d[v]<f[v]<f[u]

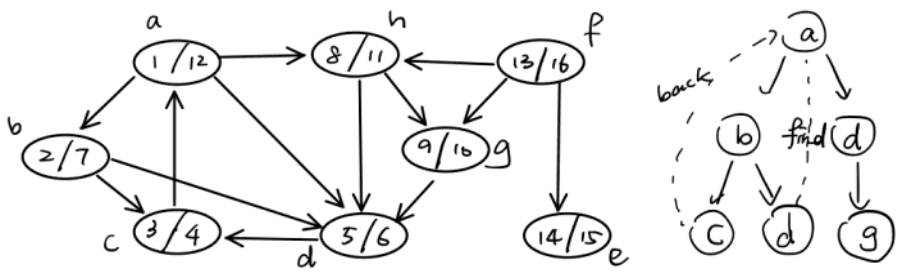
- *v descendant u back-edge*

  d[v]<d[u]<f[u]<f[v]

- **White-path theorem**
  V is a descendant of u if and only if at time
  d[u]
  , there is a path from u to v sonsisting of only white vertices

  Other edges: cross edge and tree edge

- Example:



4. Topological sort

- Directed acyclic graphs (DAG)
- Good for modeling processes and structures that have a partial order
- Topological sort:
- Linear ordering of vertices such that if
  (u,v)∈E,
  then u appears somewhere before v.
- Dependencies of getting ready

Algorithm 2 TOPOLOGICAL-SORT(V,E)
Input: A graph G:=(V, E) 1: Call DFS to compute finishing times f[v] 2: Output vertices in order of decreasing finish times

5. Number Theory

- d|a:
  d
  divides a or d is a divisor of a, a is multiple of d
- Prime number and composite number
- For and integer a and n>0, there are unique numbers q and
  0≤r≤n, such that a=qn+r(r≡a mod n)

- gcd(a,b):largest ommon divisor of a and b

- lcm(a,b):least common multiple of a and b

"put your name here" theorem
If a and b are any non-zero integer, then

$\gcd(a,b)=\min\{ax+by:x,y\in Z\}$

If
$\gcd(a,p)=1$ and$\gcd(b,p)=1$, then $\gcd(ab,p)=1$

$\gcd(a,b)\times \mathrm{lcm}(a,b)=a\times b$

6. Euclid algorithm

- Unique factorization
  A composite integer a can be written in exactly one way as a form of product of primes

- GCD recursion theorem

  $\gcd(a,b)=\gcd(b, a \bmod b)$

Algorithm GCD(a,b)
1. If b=0 then 2. return a 3. else 4. return GCD(b, a mod b) 5. end if